KnowFlow Project Blueprint

Planning baseline | 9 October 2026 | Version 1.0

This document establishes the product and engineering plan for KnowFlow. It is a design baseline for review, not an implementation specification. Decisions explicitly locked by the project owner are marked **Locked**. Recommendations that still need validation are marked **Proposed**. No application code is included.

Executive summary

KnowFlow is a workspace-based business knowledge platform that helps teams organize internal documents and find reliable answers grounded in those documents. Members upload PDFs into knowledge areas, then search or ask questions and receive answers with structured document and page citations. When the available evidence is not sufficient, KnowFlow says so instead of inventing an answer.

The core engineering challenge is building a secure and dependable product around document ingestion and retrieval: users and workspaces, authorization, relational data, background processing, search, citations, testing, deployment, and observable failure handling. AI is an integration within that system, not the product's only feature.

Locked V1 stack and direction: modular monolith; Next.js and TypeScript; Python and FastAPI; PostgreSQL with pgvector; multiple users and workspace memberships; Owner and Member roles; knowledge areas; backend-enforced tenant isolation; keyword plus semantic search; grounded answers with structured citations and insufficient-evidence handling. Document comparison and change intelligence are future work.

1. Project overview

Product and problem

Organizations accumulate policies, procedures, handbooks, technical guidance, and operational manuals in folders and shared drives. People often do not know which document is current or where a particular rule is stated. Searching filenames or reading long PDFs is slow, while generic chat answers cannot be trusted without evidence.

KnowFlow organizes documents by workspace and knowledge area, makes their content searchable, and answers questions using passages retrieved from that workspace's documents. The user can inspect the evidence behind an answer.

Target users and use cases

The initial users are small and medium-sized organizations with internal documentation and an owner responsible for maintaining it, plus employees who need to consult it. Example use cases:

- HR: find annual leave carry-over rules in the employee handbook.
- Engineering: locate deployment steps or API operating guidance.
- IT and Security: answer questions about device, access, or incident policies.
- Operations: retrieve a step from a process manual or incident procedure.
- Onboarding: help a new employee find the relevant internal guidance.

V1 is a document knowledge and retrieval product, not a replacement for a full document-management suite, records archive, or compliance certification system.

Why this is software engineering with AI integration

The project is primarily about modeling a multi-user product and making it reliable: tenant isolation, API boundaries, document lifecycle, durable processing, data integrity, search, failure recovery, security, tests, deployment, and maintainability. Embeddings and language models provide selected capabilities inside those boundaries. The project does not require custom model training or research into new ML methods.

Portfolio intent

KnowFlow should complement existing evidence of backend APIs, databases, Docker, CI/CD, SaaS workflows, and full-stack development. It should add demonstrable experience with Python/FastAPI, secure multi-tenancy, asynchronous document processing, PostgreSQL search/vector retrieval, grounded AI integration, and clear system design. The goal is a small number of substantial, polished projects that can be explained confidently to software engineering recruiters and interviewers.

2. Product vision

An organization creates a workspace, organizes knowledge areas, and uploads internal documents. Members search by phrase or ask a question. KnowFlow searches only documents they are allowed to access, assembles relevant passages, and presents a concise answer with citations the user can open and verify.

Example structure:

```
Acme Technologies (workspace)
|-- HR
|   |-- Annual Leave Policy.pdf
|   `-- Employee Handbook.pdf
|-- Engineering
|   |-- Deployment Procedure.pdf
|   `-- API Documentation.pdf
|-- IT & Security
|   `-- Information Security Policy.pdf
`-- Operations
    `-- Incident Management Procedure.pdf
```

Example question: "What does the annual leave policy say about carrying leave into the following year?" Expected behavior: retrieve passages from documents in the selected workspace, answer only to the extent those passages support, and show document title and page citations. If there is no usable evidence, explain that the workspace documents do not provide enough information and suggest refining the query or checking the source library.

KnowFlow is more than "upload a PDF and chat" because it includes workspaces and membership, organized knowledge areas, a document lifecycle with processing status and recovery, searchable document metadata and text, access-controlled retrieval, traceable citations, explicit abstention, deletion cleanup, and tests for these behaviors.

3. User personas and journeys

Workspace Owner

The Owner creates and manages a workspace, organizes knowledge areas, uploads and removes documents, and manages membership. The Owner is responsible for the workspace's content and basic access. V1 roles are intentionally simple: Owner and Member. The initial membership flow may be limited to adding an existing registered user or a similarly controlled mechanism; a full email invitation workflow is not required.

Journey: register/sign in → create workspace → create "HR" knowledge area → upload leave policy → monitor extraction status → ask a test question → verify citations → add a teammate as Member.

Workspace Member

The Member can access only workspaces they belong to. They can browse knowledge areas and documents, search, ask questions, inspect citations, and view document details. In V1 they cannot change membership or workspace settings; whether Members can upload is a product decision to confirm in detailed V1 design. Proposed default: Members can search and ask, while only Owners manage content and membership, keeping accountability simple.

Journey: sign in → select assigned workspace → open Engineering → search "deployment rollback" → review matching passages → ask a follow-up question → open a cited page to confirm the procedure.

4. Full feature roadmap

Phase	Scope	Why it belongs here
MVP / V1	Authentication; multi-user workspaces and memberships; Owner and Member roles; knowledge areas; PDF upload, validation and storage; document metadata and processing status; extraction with page provenance; chunking and embeddings; PostgreSQL/pgvector storage; basic PostgreSQL keyword search plus semantic search; grounded Q&A with structured source/page citations; insufficient-evidence response; deletion and derived-data cleanup; Docker local setup; automated tests; CI; initial cloud deployment	Delivers one coherent, usable journey while proving core software engineering and AI integration. Keeps authorization and data boundaries foundational.
V2	Invitations and membership lifecycle improvements; stronger role permissions if validated; categories and tags; conversation history; richer audit events; robust asynchronous job queue/retries and worker scaling; document versioning; operational dashboards and improved observability	Builds on a validated ingestion-and-answer loop and real usage needs. Version history is useful but adds lifecycle, storage, and retrieval semantics that should not delay V1.
V3 / future	Version comparison and change intelligence; analytics; richer integrations/imports; advanced search filters; organization-level administration; optional OCR for scanned documents; more advanced evaluation and configurable retrieval	These are valuable extensions but need proven product demand and reliable V1 foundations. Comparison and change detection explicitly remain outside MVP.

5. V1 scope and end-to-end behavior

V1 is complete when a second registered user can be given access to a workspace and both authorized users can find evidence from its PDFs without being able to observe another workspace's data.

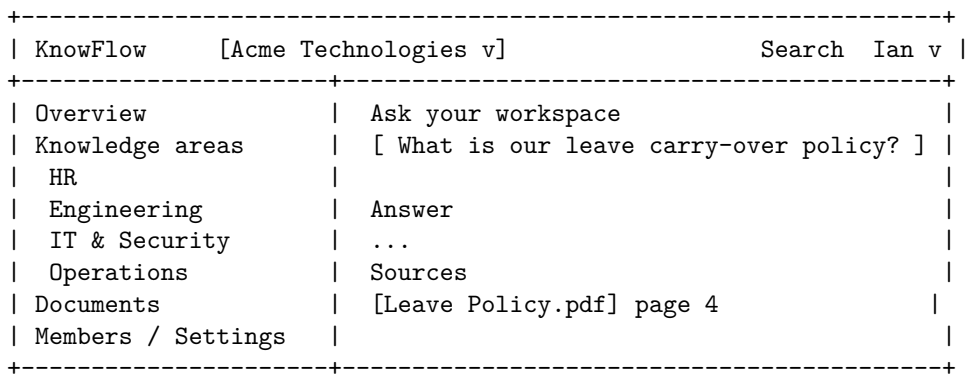
1. A user registers or signs in through the selected authentication mechanism.
2. The user creates a workspace and is assigned Owner membership.
3. The Owner creates a knowledge area, such as HR.
4. The Owner uploads a PDF. The API validates type, size, and basic file safety, stores the original, records metadata, and creates a processing record.
5. A worker extracts text page by page, retains page provenance, chunks text, requests embeddings, and stores chunks and vectors scoped to the document/workspace.
6. The document transitions through a visible status and becomes searchable only after successful processing.
7. An authorized member searches by keyword or asks a question. The backend derives workspace access from the authenticated user and filters retrieval accordingly.
8. The system combines keyword and semantic candidates, ranks/limits evidence, constructs bounded context, and asks the configured LLM for an answer constrained to that evidence.
9. The API returns answer text, an evidence/abstention status, and structured citations containing document identity, page, and passage reference. The UI renders citations as links to document detail/page context.
10. When evidence falls below a defined threshold or the model cannot support a response from the supplied passages, the system returns an insufficient-evidence outcome.
11. The Owner may delete a document. The original file and derived text/chunks/embeddings are removed or made inaccessible through a deliberate, retryable cleanup flow.

Proposed V1 limit: text-based PDFs; OCR for image-only scans is deferred unless a concrete user need justifies

its complexity. File size, page count, embedding provider, LLM provider, and evidence thresholds are configuration/design decisions for the detailed specification.

6. Screens and application layout

The application should favor a consistent workspace shell: left navigation for workspace/knowledge areas, a top bar for search and account/workspace selection, and a main content region. Keep the core search and Q&A interaction easy to reach.

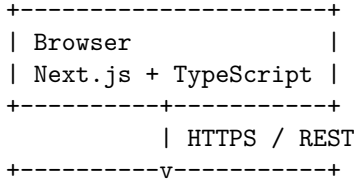


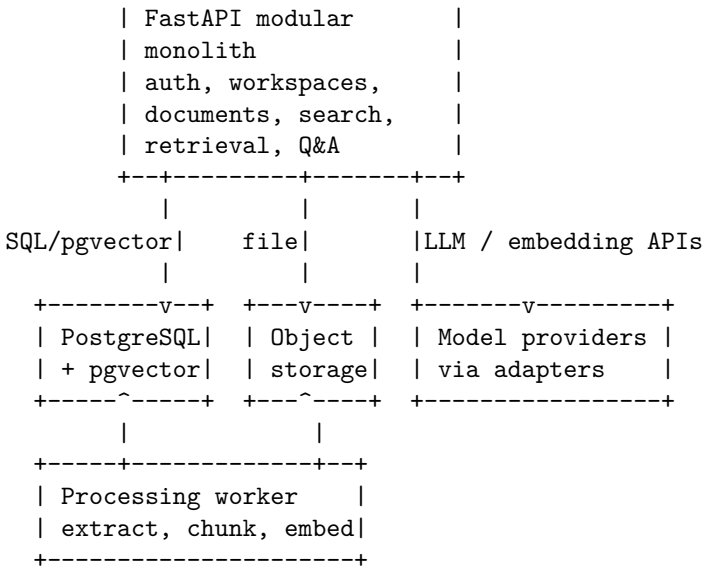
Screen	Main content and behavior
Authentication	Sign in/register, validation and error states; account recovery depends on provider choice.
Workspace creation/selection	Create a workspace, list memberships, select active workspace.
Dashboard	Workspace summary, knowledge areas, recent documents and processing failures needing attention.
Knowledge areas	List/create areas; show document counts and navigate into an area.
Document library	Search/filter by title, area, status, uploader and dates as scope permits; upload and delete controls by role.
Upload flow	Choose a knowledge area, select PDF, display validation, progress/confirmation, and processing status.
Processing status	Show uploaded/processing/ready/failed, safe failure details, retry when permitted, and timestamps.
Document detail/view	Metadata, processing state, extracted text or page preview if supported, and source context.
Search	Keyword and semantic results with snippets, area/document and page references.
Ask KnowFlow	Question input, answer, explicit insufficient-evidence state, structured citations, and optional follow-up.
Members/settings	Workspace name and member list/roles; Owner-only controls. No elaborate invitation center in V1.

Wireframe is illustrative; exact navigation, responsive behavior, PDF viewer, and membership UX are to be finalized in the V1 design.

7. System architecture

Locked: modular monolith, Next.js/TypeScript, FastAPI/Python, PostgreSQL/pgvector, backend-enforced authorization. The API and worker may be separate runtime processes built from the same backend codebase. This gives long-running work a reliable execution boundary without creating independently deployed microservices.






```
GitHub Actions: lint/typecheck/tests/build -> container images -> deploy
Docker Compose (local): frontend + API + worker + PostgreSQL (+ optional storage emulator)
```

The API owns application rules and authorization. PostgreSQL is the system of record. Original files live in object/file storage with database metadata and keys. The worker receives durable work records and updates processing state. Provider-specific LLM and embedding calls sit behind small interfaces so vendor changes do not leak throughout business logic. CI validates changes; deployment can begin with a small managed application runtime and managed PostgreSQL. Azure is a reasonable candidate given the owner's Azure familiarity, but should be selected based on cost, operational simplicity, and service fit rather than résumé keywords.

8. Document processing architecture

```
Upload -> validate -> store original -> record document/job
   -> extract per-page text -> normalize -> chunk with page provenance
   -> generate embeddings -> persist chunks/vectors -> mark READY
             | any recoverable/unrecoverable error
             +-> mark FAILED + safe error + retry policy
```

Suggested states are `UPLOADED`, `PROCESSING`, `READY`, and `FAILED`. State transitions should be explicit and transactional where practical. A document is searchable only when its processing output is complete. Processing jobs should be idempotent: retries must not create duplicate chunks or vectors. Record attempt count, timestamps, and sanitized failure category. Retry transient provider/network errors with bounded backoff; do not endlessly retry invalid or unsupported files.

Retain original file identity, checksum, size, content type, uploader, workspace, knowledge area, and timestamps. Each extracted page and chunk must preserve its source document and page range. On reprocessing, replace derived records atomically or stage new output before switching the document to READY. Deletion must prevent further access immediately, then clean the file and derived records; cleanup failure should be observable and retryable. Retention and soft-delete behavior need a final product decision.

9. Search and RAG architecture

1. **Ingestion and embeddings:** extracted text is split into bounded, overlapping chunks with stable document/page provenance. An embedding provider converts each chunk into a vector stored in pgvector. Provider/model and vector dimensions are recorded/configured consistently.
2. **Keyword search:** PostgreSQL full-text search supports exact policy terms, names, acronyms, and numbers. It provides a practical baseline and complements semantic matching.
3. **Semantic search:** embed the user's query using the same embedding model and retrieve nearest chunks from pgvector. Similarity is a relevance signal, not proof that a passage answers the question.
4. **Authorization and retrieval:** establish authorized workspace IDs from the session and membership table before querying. Apply workspace filters to both keyword and vector paths in the database query. Combine candidates with a simple, explainable ranking method in V1; avoid a separate search service initially.
5. **Context construction:** deduplicate and cap the strongest passages; include document title, page, and chunk identifiers. Enforce token/context limits and avoid sending unrelated content.

6. **Answer generation:** ask the LLM to answer only from the supplied passages, preserve uncertainty, and return a structured response. The model is not trusted to invent citation IDs.
7. **Citation rendering:** validate model-referenced source IDs against the retrieved set and return citations from server-owned metadata (document, page, snippet). Unknown citation IDs are discarded or cause a safe fallback.
8. **Insufficient evidence:** use retrieval score/coverage checks plus a constrained answer format. If evidence is too weak or the answer lacks support, return a clear abstention. Thresholds must be evaluated, not guessed permanently.

Uploaded text is untrusted data. It may contain instructions aimed at the model. Treat retrieved passages only as quoted evidence, tell the model to ignore instructions contained in source material, limit tools/capabilities (ideally none for V1), and never let model output bypass authorization or select arbitrary files. Prompting is not a security boundary; tenant filtering, input/output validation, and constrained capabilities are.

10. Database design

Proposed conceptual ER diagram (exact normalization and provider fields are to be finalized before migrations):

```
users 1---* workspace_memberships *---1 workspaces
                                       |
workspaces 1---* knowledge_areas
workspaces 1---* documents *---0..1 knowledge_areas
users 1---* documents (uploaded_by)
documents 1---* processing_jobs
documents 1---* document_pages 1---* document_chunks
document_chunks 1---0..1 chunk_embeddings (or vector column on chunks)
workspaces 1---* conversations 1---* questions (optional V1)
questions 1---* answer_citations *---1 document_chunks
```

Entity	Purpose and key relationships
`users`	Application identity or provider-linked profile; unique provider subject/email as appropriate.
`workspaces`	Tenant boundary and organization container; created by an Owner.
`workspace_memberships`	Many-to-many user/workspace join, role (`OWNER`/`MEMBER`), status and timestamps; unique `(workspace_id, user_id)`.
`knowledge_areas`	Workspace-owned organizational grouping; unique name per workspace if desired.
`documents`	Workspace and area ownership, uploader, storage key, metadata, processing state, checksum and timestamps.
`processing_jobs`	Durable attempts/status/error category and retry metadata; tied to document and workspace.
`document_pages`	Extracted page text and page number, or equivalent provenance records if extraction design stores text elsewhere.
`document_chunks`	Chunk text, order, page range, token/character counts and provenance.
`chunk_embeddings`	Optional separate vector table keyed to chunk and model/version; alternatively vector column on chunks for a single configured model.
`conversations`, `questions`, `answer_citations`	Optional history. V1 can return answers without persistence; if persisted, preserve workspace/user ownership and citation links.

Core indexes: membership lookup on `(user_id, workspace_id)` and `(workspace_id, role)`; documents on `(workspace_id, knowledge_area_id, status)` and `(workspace_id, created_at)`; unique document checksum scoped as product rules require; chunks on document/page; PostgreSQL full-text index on searchable text; pgvector index selected after measuring dataset and query needs. Use foreign keys and unique constraints to encode ownership and prevent duplicate membership. Avoid premature partitioning or elaborate schemas.

Tenant considerations: workspace ID should be present or derivable for every tenant-owned record. Where practical, use composite keys/foreign keys that ensure a chunk cannot point to a document from another workspace. Every query path (including status, deletion, citations, and worker updates) must validate workspace/document ownership. PostgreSQL row-level security is a possible defense-in-depth option, but adds connection/session complexity; decide after implementation constraints are clear rather than assuming it is mandatory.

11. Multi-tenancy and authorization

Multiple users and memberships are **Locked for V1**. A workspace has an Owner and Members; a user can belong to more than one workspace. Ownership and roles are represented in `workspace_memberships`, not inferred from a client-provided workspace ID.

For each request, authentication identifies the user. A backend authorization dependency/service loads membership for the requested workspace and verifies the action against the role. Data access functions accept an authorized workspace context and always scope reads/writes by it. Search retrieves candidates only after applying workspace scope; citations are validated against those candidates; generated answers are assembled only from authorized passages. The model never receives other tenant data. Workers verify document/workspace association before processing and deleting.

Add negative tests with two workspaces and overlapping users to prove that one user cannot enumerate, fetch, search, cite, delete, or infer another workspace's documents or chunks. Return consistent not-found/forbidden behavior to avoid leaking resource existence. Membership creation and transfer/owner deletion policy should be finalized in detailed V1 design; maintain at least one Owner per active workspace.

12. REST API design

Illustrative resource API; exact payloads, pagination, errors, and auth transport are for the V1 specification.

Area	Example endpoints
Authentication	`POST /auth/register`, `POST /auth/login`, `POST /auth/logout`, `GET /auth/me` (or provider-managed equivalents)
Workspaces	`GET /workspaces`, `POST /workspaces`, `GET /workspaces/{workspace_id}`, `PATCH /workspaces/{workspace_id}`
Memberships	`GET /workspaces/{workspace_id}/members`, `POST /workspaces/{workspace_id}/members`, `PATCH /workspaces/{workspace_id}/members/{user_id}`, `DELETE .../{user_id}`
Knowledge areas	`GET/POST /workspaces/{workspace_id}/knowledge-areas`, `GET/PATCH/DELETE /workspaces/{workspace_id}/knowledge-areas/{area_id}`
Documents	`GET/POST /workspaces/{workspace_id}/documents`, `GET/DELETE /workspaces/{workspace_id}/documents/{document_id}`, optional `GET .../{document_id}/content`
Processing	`GET /workspaces/{workspace_id}/documents/{document_id}/statu` `POST .../{document_id}/retry` (Owner-only or policy-defined)
Search	`POST /workspaces/{workspace_id}/search` with query, filters and limit; response includes ranked snippets and source/page metadata
Questions	`POST /workspaces/{workspace_id}/questions` with question and optional area filter; response includes answer status, answer and structured citations

Use consistent validation and error envelopes, bounded pagination/results, request IDs, and OpenAPI documentation. Consider signed upload URLs only if direct object-store uploads are needed; a backend-mediated upload is simpler for a small V1, subject to configured size limits.

13. Security baseline

- **Authentication:** choose a maintained provider/library. Proposed direction: managed OIDC/auth provider to avoid building password reset, verification, session rotation and abuse controls; alternatively use a proven framework component if self-hosting is required. Avoid custom password cryptography. Final provider depends on cost, deployment and user ownership requirements.
- **Sessions:** secure, HttpOnly, SameSite cookies for browser sessions where supported; CSRF protections for cookie-authenticated mutations; short-lived tokens and refresh strategy if using bearer auth.
- **Authorization:** server-side membership and role checks for every tenant resource and operation. UI hiding is not authorization.
- **Uploads:** allow only PDF, enforce size/page limits, verify content rather than trusting filename/MIME, generate storage keys, sanitize displayed names, and isolate uploaded files from executable paths. Consider malware scanning if deployment exposure warrants it.
- **Secrets:** environment/secret manager only; never commit provider credentials. Keep separate development/test/production configuration and rotate exposed secrets.
- **Validation and abuse:** typed request schemas, bounded payloads, safe error messages, upload and question rate limits, timeouts, and quotas when exposed publicly.
- **Document trust boundary:** extracted content may be malicious or misleading. It is data, not executable instructions. Do not allow documents to trigger tools or affect access control.
- **LLM boundaries:** minimize submitted personal data, avoid sending unnecessary pages, constrain output schema, validate citations, log metadata rather than sensitive full prompts by default, and provide provider error handling.
- **Database/storage:** least-privilege credentials, encrypted transport, backups, private object storage, and retention/deletion policy. Ensure logs do not expose document content or tokens.

14. Testing strategy

Test layer	Coverage
Unit	Role policy, workspace context, file validation, state transitions, text normalization, chunk boundaries/provenance, ranking merge, abstention rules, citation validation.
API/integration	Register/login adapter behavior; workspace/area/document CRUD; processing status; search/question contracts; database constraints and migration behavior.
Authorization/tenant isolation	Two-user/two-workspace matrix; cross-tenant reads, updates, deletes, search, citations and worker actions all denied without leaking content.
Processing	Valid PDF extraction by page, malformed/empty/large PDFs, duplicate/retry idempotency, embedding provider failure, cleanup and retry. Use deterministic fixtures and mocked external services.
Retrieval/citations	Known queries retrieve expected source passages; returned citations are server-validated, point to correct document/page, and never cite an unreturned chunk. Include no-evidence cases.
End-to-end	One focused browser path: owner creates workspace/area, uploads fixture, waits for ready, asks question, opens citation; member access and unauthorized workspace denial. Avoid a large brittle UI suite.
Evaluation set	Maintain a small fixed set of representative questions, expected source documents/pages/passages, and expected abstentions. Track whether retrieval includes expected evidence and whether citation/answer behavior regresses.

External model calls should not make ordinary tests nondeterministic or costly. Mock provider interfaces in unit/integration tests; run a small opt-in evaluation against real providers when tuning prompts or retrieval.

15. Development environment and repository structure

Proposed monorepo layout:

```
knowflow/
|-- apps/
|   `-- web/                    # Next.js app, routes, UI components
|-- backend/
|   |-- app/
|   |   |-- api/                # FastAPI routers and schemas
|   |   |-- core/               # settings, auth, errors, logging
|   |   |-- modules/            # workspaces, membership, areas, documents, search
|   |   |-- processing/         # extraction, chunking, embeddings, worker
|   |   |-- retrieval/          # keyword/vector fusion, context, answer adapter
|   |   `-- db/                 # models, repositories, migrations
|   `-- tests/                  # unit, integration, fixtures, evaluations
|-- docs/                       # blueprint and evolving design records
|-- infra/                      # deployment definitions/configuration as justified
|-- docker-compose.yml
|-- .env.example
`-- .github/workflows/          # CI and deployment workflows
```

This keeps a single deployable backend codebase while making domain boundaries visible. Avoid splitting into extra packages/services until a real ownership, scaling, or release need exists.

Docker Compose should provide frontend, API, worker, PostgreSQL with pgvector, and a local file/object storage option if needed. One configuration path should work for a new developer. `.env.example` documents variable names without secrets; validate required settings at startup. Keep migrations explicit and run them in a controlled step rather than silently on every application process.

16. CI/CD and deployment

CI proposal

On pull requests: formatting/lint, frontend type-check and build, backend lint/type checks, unit tests, integration tests against ephemeral PostgreSQL/pgvector, migration validation, and container build. Cache dependencies safely. Require checks before merging. Keep a separate workflow for dependency/security scanning if it remains maintainable.

Deployment proposal

Start with a small managed container runtime for web/API/worker and managed PostgreSQL with pgvector, plus private object storage and managed secrets. Deploy a single region initially. Apply database migrations as a reviewed deployment step; use health/readiness checks and retain rollback ability. Store logs and basic metrics/errors with request IDs and job IDs. Azure Container Apps or an equivalent simple container platform could host the app, with Azure Database for PostgreSQL and Blob Storage if service availability, pgvector support, cost, and operational needs fit. Verify exact current service support at deployment design time.

Do not choose Kubernetes, service mesh, Kafka, or complex cloud networking for a portfolio V1. A reliable small deployment with backups, environment separation, and observable worker failures is stronger evidence.

17. Implementation plan and acceptance milestones

Milestone	Objective / components	Expected output	Required tests	Acceptance criteria
0. Design baseline	Confirm product boundaries, auth choice, upload limits, provider strategy, data-retention and membership rules	Reviewed V1 specification, flows, ERD, threat notes	Design walkthrough against use cases and tenant boundaries	No unresolved decision blocks schema or first vertical slice; owner approves plan before code begins

Milestone	Objective / components	Expected output	Required tests	Acceptance criteria
1. Foundation	Monorepo, frontend/backend skeletons, configuration, Docker Compose, PostgreSQL/pgvector, migrations, CI checks	Reproducible local environment and green CI	Startup/config tests, DB connection and migration smoke test	New setup can start all required components from documented steps
2. Identity and tenancy	Auth integration, users, workspaces, memberships, roles, authorization dependencies	Users can create/select workspaces; Owner/Member enforced	Auth integration tests and cross-tenant negative matrix	A user sees only memberships; workspace APIs enforce role and membership
3. Knowledge areas and document management	Area/document metadata, upload validation/storage, list/detail/delete, status model	Upload and manage PDFs with visible lifecycle	Validation, CRUD, deletion authorization and storage tests	Invalid files rejected; authorized uploads are recorded and inaccessible across tenants
4. Processing pipeline	Worker/job records, extraction, page provenance, chunking, embeddings, retries	Documents move to READY or FAILED with inspectable status	Fixtures, idempotency, provider failure/retry, cleanup tests	READY means searchable derived data is complete; retry does not duplicate output
5. Search and retrieval	PostgreSQL keyword search, pgvector semantic search, tenant filters, fusion/ranking	Authorized search results with snippets and provenance	Retrieval evaluation set and tenant isolation	Representative queries return expected passages; exact terms and semantic queries both work
6. Grounded Q&A	Context construction, LLM adapter, structured answer/citations, abstention	Ask KnowFlow returns supported answer or insufficient-evidence state	Citation integrity, unsupported question and prompt-injection fixtures	Every citation maps to retrieved authorized evidence; unsupported requests abstain
7. Product completion and security	Main screens, role-aware UX, logs/error states, rate limits and security review	Coherent usable end-to-end product	API, browser E2E path, authorization regression suite	Owner and Member flows work; failure states are actionable and safe
8. Release and deployment	Container pipeline, environment/secrets, managed DB/storage/app runtime, backups/health checks	Public or controlled demo deployment	Deployment smoke tests, migration/rollback check, upload-to-answer smoke test	Deployed journey works; backups and operational failure visibility are documented
9. V2 improvements	Based on use and feedback: history, invitations, tags, audit, versioning, job queue improvements	Prioritized next release	Feature-specific tests and updated evaluation	Each addition addresses observed need and preserves tenant guarantees

Implementation should proceed vertically: first create the smallest secure end-to-end slice, then deepen reliability and product polish. The exact sequence may adjust after the detailed V1 design, but tenant boundaries and data contracts must precede document retrieval.

18. Portfolio and interview value

Audience	Evidence KnowFlow can provide
Software Engineer	Clear domain boundaries, API contracts, validation, state machines, tests, error handling, maintainable changes and trade-off reasoning.
Backend Engineer	Relational modeling, migrations, membership authorization, tenant isolation, background processing, retries/idempotency, search and operational behavior.
Full-Stack Engineer	User journeys and screens connected to secure REST endpoints, loading/error/processing states, responsive interface, integration and deployment.
Internship/graduate roles	A substantial project with a coherent problem, documented decisions, demonstrable deployment, tests, and a grounded explanation of what was learned and improved.

Interview decisions to understand well:

- Why a modular monolith fits the project's scale and how modules communicate.
- How workspace membership is checked and why client-supplied IDs cannot grant access.
- How document processing states, retries and idempotency protect data integrity.
- Why PostgreSQL keyword search and pgvector complement one another.
- How page provenance survives extraction and chunking to produce trustworthy citations.
- How the system detects insufficient evidence and why a language model is not the authority for access or citation metadata.
- How deletion and reprocessing handle original and derived data.
- How tests prove tenant isolation and retrieval quality.
- What would justify adding Redis/job queue, OCR, a dedicated search service, or a different deployment topology later.

19. Explicit non-goals

V1 deliberately does not include:

- Microservices, Kubernetes, service mesh, Kafka, or distributed infrastructure without a demonstrated need.
- Complex multi-agent AI systems or autonomous tools.
- Custom model training, advanced ML research, or a mathematics-heavy ML focus.
- Document version comparison or change intelligence (later phase).
- Enterprise-grade granular permissions before a real use case exists.
- A full email invitation and organization provisioning system.
- OCR for every scanned format, broad file-type support, or a complete records-management suite.
- A separate search cluster or vector database when PostgreSQL can support the initial workload.
- Analytics and complex administration before the core retrieval workflow is dependable.

20. Final visual roadmap

```
PLANNING
   -> FOUNDATION
   -> AUTHENTICATION / MULTI-TENANCY
   -> DOCUMENT MANAGEMENT
   -> PROCESSING PIPELINE
   -> SEARCH / RETRIEVAL
   -> RAG / CITATIONS
   -> TESTING / SECURITY
   -> DOCKER / CI/CD
   -> DEPLOYMENT
   -> V2 IMPROVEMENTS
```

The sequence represents dependencies and a reviewable progression, not a promise of fixed dates. The first release is complete when the owner/member journey works end to end, answers are traceable or withheld when unsupported, and tenant isolation is demonstrated by tests.

Decisions locked and decisions for detailed V1 design

Locked by project owner: modular monolith; Next.js/TypeScript; Python/FastAPI; PostgreSQL/pgvector; multiple workspace users/memberships from the beginning; Owner and Member roles; no elaborate invitations initially; knowledge areas; backend tenant isolation; ingestion-to-grounded-answer/citation workflow; keyword search alongside semantic; insufficient-evidence behavior; version comparison/change intelligence deferred.

Proposed for detailed design: managed authentication rather than custom password handling; Owner-only content/membership management with Members primarily searching and asking; PDF-only text extraction for V1; no persisted conversation history until justified; durable database job records and a worker process before adding Redis; object storage for deployed originals; no OCR in V1; a simple hybrid ranking method and measured abstention thresholds. These should be confirmed or adjusted when we prepare the detailed V1 product specification.

Blueprint prepared for review before implementation. Update this source alongside the PDF as product and architecture decisions evolve.